

A Deep Dive into Rainbow DQN

Chapter 5: Distributional Reinforcement Learning

Harsh Verma

June 26, 2025

Abstract

Previous enhancements to DQN focused on improving the accuracy and stability of learning the *expected* return. Distributional Reinforcement Learning represents a paradigm shift, proposing that the agent should learn the full *distribution* of possible returns for each action. By modeling this distribution of outcomes, the agent can represent value uncertainty, mitigate the negative effects of multimodality, and learn a richer, more stable value function. This chapter details the theory behind the Distributional Hypothesis and its practical implementation via the Categorical DQN (C51) algorithm.

1 The Motivation: The Flaw of Averages

A standard Q-Network learns to predict a single scalar value for each action: the statistical mean (or expectation) of the discounted future rewards. However, this average can be a misleading and incomplete representation of reality.

Hypothesis 1.1 (The Distributional Hypothesis). *The value of a state-action pair is not a single number, but a distribution of random returns, referred to as the value distribution, $Z(s, a)$. Learning this full distribution, rather than just its expectation $Q(s, a) = \mathbb{E}[Z(s, a)]$, leads to more stable learning and better policies.*

Consider two actions with the same expected return of +5:

- **Action A (Safe):** Guarantees a reward of +5. The distribution is a single spike at +5.
- **Action B (Risky):** Has a 50% chance of +20 and a 50% chance of -10. The distribution is bimodal, with peaks at +20 and -10.

A standard DQN is blind to this difference. A distributional agent can perceive the risk and potentially learn to prefer the safer action, even if their means are identical.

2 The Categorical Approach (C51)

Modeling a continuous distribution is computationally difficult. The Categorical DQN algorithm (Bellemare et al., 2017), often called C51 for its use of 51 support points ("atoms"), proposes a practical solution by discretizing the distribution.

2.1 From Scalar to PMF

1. **Define a Support:** A fixed range of possible returns, $[V_{\min}, V_{\max}]$, is defined as a hyper-parameter.

2. **Create Atoms:** This range is discretized into a set of N equally spaced points, called atoms (z_i). These atoms form the support of our distribution.
3. **The Network's New Role:** The network no longer outputs a single Q-value per action. Instead, for each action, it outputs a vector of probabilities, p_i , one for each atom. This vector represents a Probability Mass Function (PMF).

The network's output for an action a is thus a distribution: $d_t(s, a) = \{(z_i, p_i(s, a))\}_{i=1}^N$.

3 The Distributional Bellman Update

Training this network requires a new target and a new loss function. The target is no longer a scalar but another distribution, and the loss measures the "distance" between the predicted and target distributions.

3.1 The Projection Step: A Critical Hurdle

The Distributional Bellman Operator, $\mathcal{T}$, is applied to each atom z_j of the target network's distribution for the next state-action pair (s', a^*):

$$\mathcal{T}z_j = r + \gamma z_j$$

This operation discounts and shifts the support of the target distribution. The problem is that these new support points, $\mathcal{T}z_j$, will not align with our original, fixed atoms z_i . We cannot directly compare the two distributions.

The solution is a **projection step**: for each updated target atom $\mathcal{T}z_j$, we distribute its probability mass $p_j(s', a^*)$ to its immediate neighbors on our original support grid. This creates a new target distribution, m , that lives on the same support as our prediction.

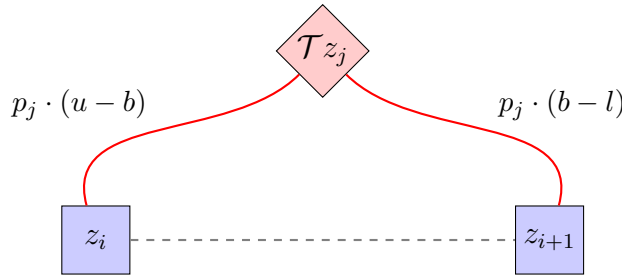


Figure 1: A diagram illustrating the projection step for a single atom from the target distribution. The probability mass of each updated atom from the target distribution (center) is projected onto its two nearest neighbors in the fixed support grid (bottom). This creates a new target distribution that can be directly compared to the network's prediction.

3.2 The Loss Function: Comparing Distributions

With two distributions on the same support (our prediction p and the projected target m), we can compute the distance between them. The appropriate loss function is the **Kullback-Leibler (KL) Divergence**, which for a discrete distribution is equivalent to minimizing the cross-entropy:

$$L(s, a) = - \sum_{i=1}^N m_i \log(p_i(s, a))$$

This loss pulls the predicted distribution towards the target distribution, providing a rich, multi-point learning signal for every update.

4 Implementation and Advanced Topics

4.1 PyTorch Implementation Snippet

The implementation requires a modified network head and a custom loss function.

```
1 # online_net, target_net, transitions, etc. are assumed to be defined
2
3 # Get the log-probabilities from the online network for the chosen actions
4 log_p = online_net(states)
5 log_p_a = log_p[range(BATCH_SIZE), actions]
6
7 with torch.no_grad():
8     # --- Form the target distribution ---
9     # Get next state distributions and select best actions (DDQN style)
10    p_next = torch.exp(target_net(next_states))
11    q_next_online = (torch.exp(online_net(next_states)) * atoms).sum(2)
12    best_actions = q_next_online.argmax(1)
13    p_next_best = p_next[range(BATCH_SIZE), best_actions]
14
15    # --- Projection Step ---
16    # Bellman update for each atom
17    Tz = rewards.unsqueeze(1) + (1 - dones.unsqueeze(1)) * gamma * atoms.
18    unsqueeze(0)
19    Tz = Tz.clamp(min=V_MIN, max=V_MAX)
20
21    # Compute indices and proportions for projection
22    b = (Tz - V_MIN) / delta_z
23    l, u = b.floor().long(), b.ceil().long()
24
25    # Distribute probability mass to create target distribution m
26    m = torch.zeros_like(p_next_best)
27    m.view(-1).index_add_(0, (l + offset).view(-1), (p_next_best * (u.float() -
28    b)).view(-1))
29    m.view(-1).index_add_(0, (u + offset).view(-1), (p_next_best * (b - l.float
30    ())).view(-1))
31
32    # --- Calculate the final loss ---
33    # Cross-entropy loss between the target distribution m and predicted log-
34    probabilities
35    loss = -(m * log_p_a).sum(dim=1)
36
37    # Apply PER weights (if applicable) and average
38    final_loss = (loss * weights).mean()
```

Listing 1: Core logic for the Distributional Loss calculation.

4.2 Beyond Categorical: QR-DQN and IQN

The Categorical (C51) approach is just one way to model the value distribution. It has a key limitation: the support $[V_{\min}, V_{\max}]$ is a fixed hyperparameter. More advanced methods improve on this:

- **Quantile Regression DQN (QR-DQN):** Instead of learning the probabilities for fixed locations, QR-DQN learns the *locations* (quantiles) of the return for a fixed set of probabilities (e.g., the 5th, 50th, and 95th percentiles).
- **Implicit Quantile Networks (IQN):** This is the most advanced version, used in some state-of-the-art agents. It learns a continuous mapping from a probability $\tau \in [0, 1]$ to its corresponding return value, effectively learning the entire quantile function.

These methods provide more flexible and powerful representations of the value distribution. However, the Categorical approach remains a foundational and highly effective component of the original Rainbow agent.